به نام خدا



پروژه طراحی کامپایلر

استاد

دکتر آرش شفیعی

دستیاران آموزشی

نگار نادیان

ملیکا خندان

اعضای تیم

سید حسین حسینی

رضا اعلایی

دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

بهار ۱۴۰۳

فهرست

مقدمه 1

تحلیل گر لغوی (LEXICAL ANALYZER) 1

توابع (FUNCTIONS) 1

input identifier 1

output identifier 1

keyword identifier 2

id identifier 4

scope identifier 5

decimal identifier 6

hexadecimal identifier 7

whitespace identifier 8

const char identifier 9

operator identifier 10

comment identifier 12

مقدمه

یک کامپایلر از بخش های مختلفی شامل تحلیل‌گر لغوی، تحلیل‌گر نحوی، تحلیل‌گر معنایی ساخته شده است، که در این قسمت به نحوه ساخت و بررسی تحلیل‌گر لغوی می پردازیم.

در این پروژه قصد داریم که یک کامپایلر برای زبان **PL** بنویسیم که نیازمند پیاده سازی تحلیل گر های متفاوت است و در همین راستا به پیاده سازی آن ها می پردازیم و لازم به ذکر است که کامپایلر ها ار می توان با زبان های مختلفی پیاده کرد و زبان انتخابی ما در این پیاده سازی زبان جاوا (**Java**) می باشد.

تحلیل‌گر لغوی (Lexical Analyzer)

تحلیل گر لغوی برای درک یک زبان برنامه نویسی کاربرد بسیار دارد، بدین صورت که باید برنامه نوشته شده را به توکن هایی برای درک کامپایلر تبدیل کنیم. برای اینکه بتوانیم توکن های مورد نظر را داشته باشیم، باید از عملکرد توکنایز کردن استفاده کنیم که این کار نیازمند داشتن تحلیل گر لغوی هستیم و در ادامه به روند توکنایز کردن و روند تبدیل آن ها می پردازیم.

توابع (Functions)

برای اینکه بتوانیم انواع توکن ها استخراج کنیم، نیازمند این است برای هرکدام از آن ها یک تابع به خصوص بنویسیم تا هر کدام عملکرد درست خود را داشته باشد، زیرا روند استخراج توکن های مختلف متفاوت است و تابعی نوشتن آن ها باعث درک بهتر روند استخراج توکن می شود.

- input identifier

در این تابع قصد داریم ورودی خود را که در قالب یک فایل تکست است دریافت کنیم و این کار نیازمند بررسی فایل های موجود در دایرکتوری ورودی و سپس دریافت به نوبت آن ها در ورودی است تا بتوانیم آن ها را بررسی کنیم.

- Output identifier

در این تابع قصد داریم خروجی خود را در قالب یک فایل تکست تولید کنیم و همچنین جهت بررسی بهت نتایج را در ترمینال نیز چاپ می کنیم تا از صحت اطلاعات خروجی و نمایش دادن آن ها استفاده کنیم و همچنین از قابلیت های این تابع این است که به تعداد ورودی ها خروجی های مورد لازم را تولید می کند.

- keyword identifier

در این تابع قصد داریم توکن های کلمات کلیدی که شامل عناوین و نوع دستورات هست استخراج کنیم و برای این کار نیاز است که کاراکتر به کاراکتر ورودی را بررسی کنیم تا بتوانیم کلمات کلیدی را استخراج کنیم ولی در صورتی که در حین استخراج کلمات کلیدی متوجه شویم که کلمه مورد نظر ما که در حال پیمایش هست، برای استخراج توکن با کلمات کلیدی مطابقت نداشته باشد تا اتمام کلمه دیگر بررسی انجام نخواهد شد، و همچنین اگر در داخل رشته یا کامنت باشیم، به طور کامل بررسی نخواهیم داشت تا با کاراکتر های خط جدید یا یکسری از مواردی که ما از آن ها خارج می کنند، رو به رو بشویم و سپس به روند اصلی استخراج باز گردیم.

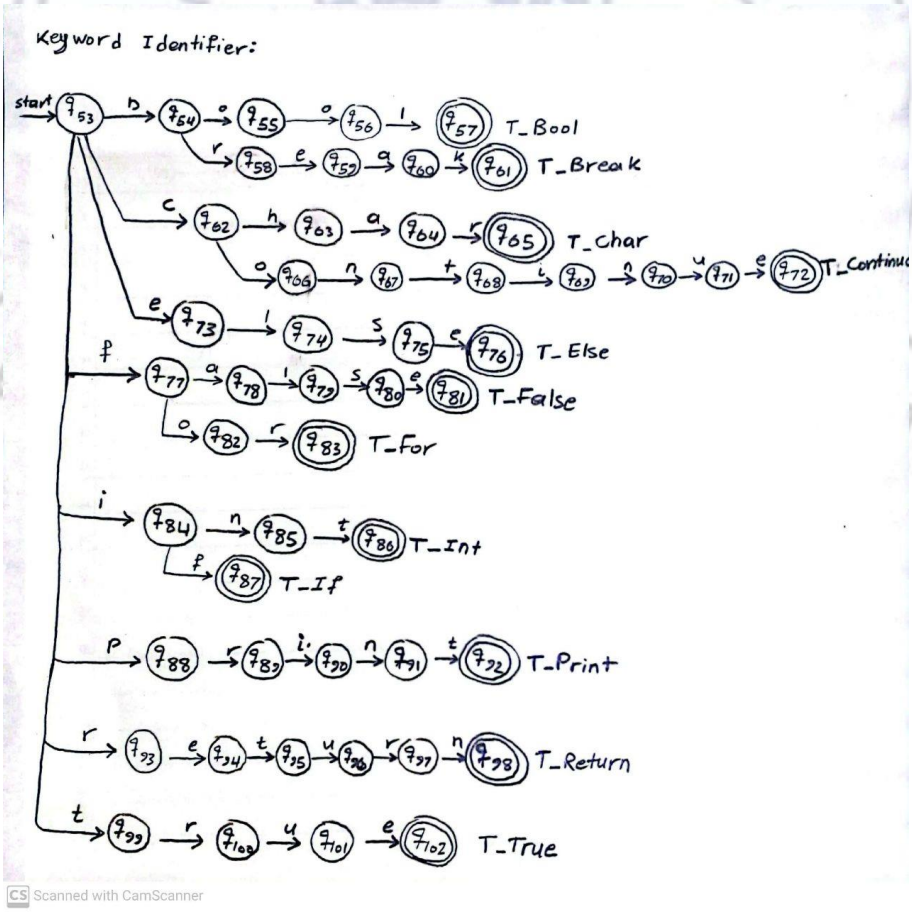
کلمات کلیدی شامل کلمات زیر است :

---

bool , break , char , continue , else , if , false , for , int , print , true , return

---

دیاگرام گذار



Scanned with CamScanner

عبارت منظم

bool

break

char

continue

else

if

false

for

int

print

true

return

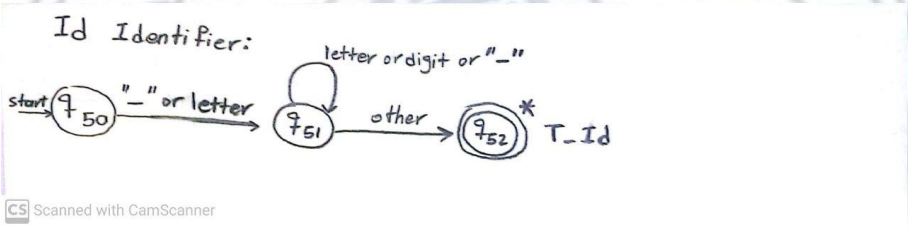
- id identifier

در این تابع قصد داریم توکن های شناسه ها را استخراج کنیم و این بخش، شامل کلمات زیادی می شود که با نام این توکن استخراج می شوند زیرا این کلمات به سلیقه و دلخواه کاربری که کد نویسی انجام می دهد انتخاب می شود و با وجود این نکته که در قالب خاصی باید این کار انجام شود که مورد اعتبار باشد و مشکلی در روند برنامه ایجاد نکند، به این دلیل که برای مثال مشابهت یا اسفاده از علامت های نامتعارف روند تحلیل لغوی را دچار مشکل خواهد کرد.

شناسه ها شامل موارد زیر می شوند :

نام متغیر ها ، نام توابع

دیاگرام گذار



عبارت منظم

(_ | letter) (_ | letter | digit) *

letter -> [A_Za_z]

digit -> [0_9]

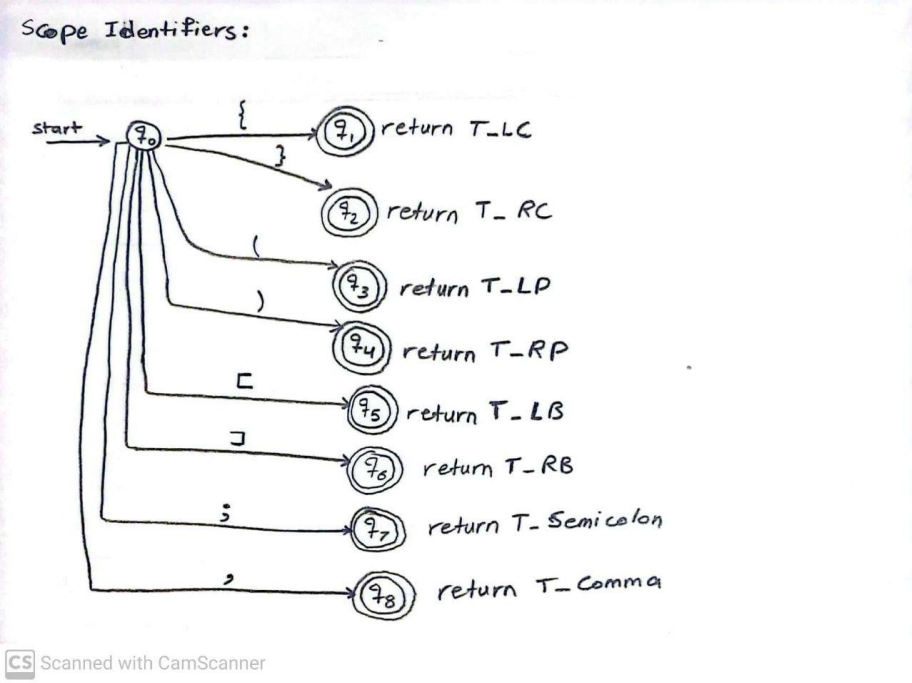
- scope identifier

در این تابع قصد داریم توکن های مربوط به علامت های نشانه گذاری را استخراج کنیم و به دلیل اینکه به صورت تک کاراکتر هست در استخراج به مشکل خاصی برنخواهیم خورد.

کلمات نشانه گذاری شامل موارد زیر می شود :

“{“ , “}” , ”(“ , “)” , “[“ , “]” , “;” , “;”

دیاگرام گذار



عبارت منظم

به دلیل اینکه به صورت تک کاراکتر هستند، عبارات منظم آن ها به همین صورت می باشد و تفاوتی با قسمت معرفی آن ها که در قسمت بالا انجام شد ندارد ولی برای اینکه متوجه آن ها شویم در صورت تعریف متغیر برای هریک نام های اصلی آن ها را می گذاریم.

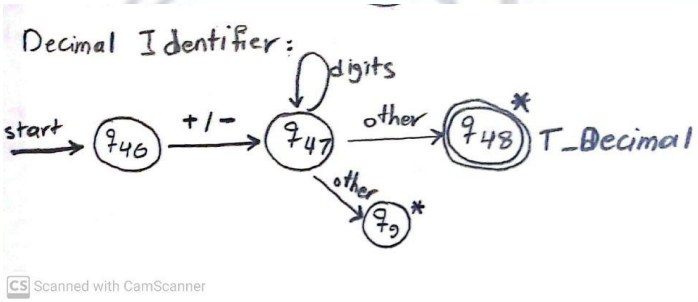
- decimal identifier

در این قسمت قصد داریم توکن های مربوط به اعداد دسیمال یا به اصطلاح دهدهی را استخراج کنیم که ابتدا مثبت و یا منفی بودن آن را مشخص می کنیم و سپس خود عدد را که در نهایت توکن استخراج می شود.

اعداد دهدهی (به همراه نوع علامت) شامل موارد زیر می شود :

- , + , 0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9

دیاگرام گذار



Scanned with CamScanner

عبارت منظم

$$(+|-)(digit)^{+}$$

$$digit \rightarrow [0_9]$$

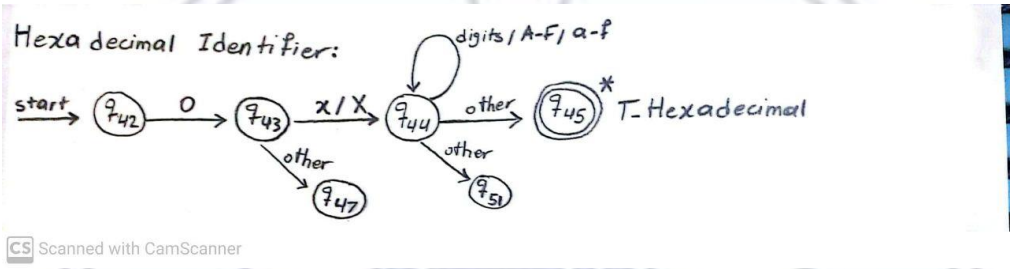
- hexadecimal identifier

در این قسمت قصد داریم توکن های مربوط به اعداد هگزا دسیمال یا به اصطلاح مبنای شانزده را استخراج کنیم که ابتدا آن ها با **0X** یا **0x** شروع می شوند، که نشان دهنده این سری از توکن ها می باشد.

اعداد مبنای شانزده شامل موارد زیر می شود :

0 , 1 , 2 , 3 , 4 , 5 , 6 , 7 , 8 , 9 , A , B , C , D , E , F

دیاگرام گذار



Scanned with CamScanner

عبارت منظم

$0\,(x \mid X)\,(digit \mid [A_F] \mid [a_f])^{+}$

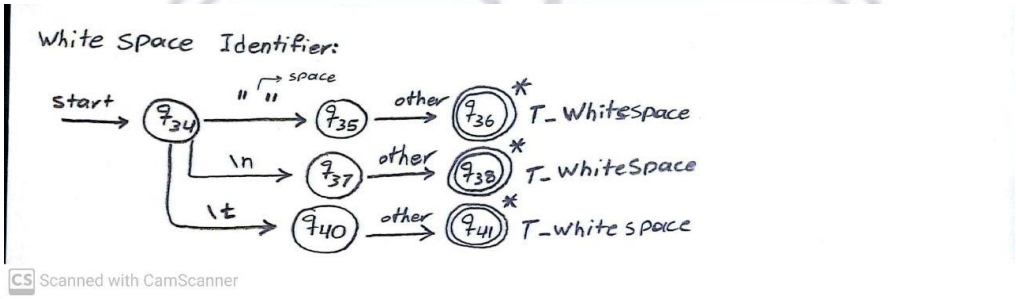
- whitespace identifier

در این قسمت قصد داریم توکن های مربوط به فضاهای خالی را که جداکننده های اصلی یک برنامه برای درک بهتر آن هستند، را استخراج کنیم.

فضاهای خالی شامل موارد زیر می شود :

" " -> Space , "\n" -> Enter , "\t" -> Tab

دیاگرام گذار



Scanned with CamScanner

عبارت منظم

به دلیل اینکه به صورت تک کاراکتر هستند، عبارات منظم آن ها به همین صورت می باشد و تفاوتی با قسمت معرفی آن ها که در قسمت بالا انجام شد ندارد ولی برای اینکه متوجه آن ها شویم در صورت تعریف متغیر برای هریک نام های اصلی آن ها را می گذاریم.

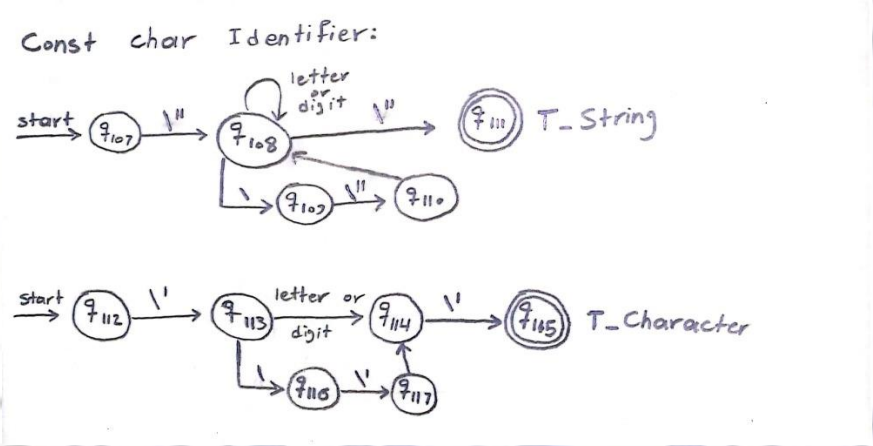
- const char identifier

در این قسمت قصد داریم توکن های مربوط به رشته ها و کاراکتر ها را استخراج کنیم تا بتوانیم آن ها را با دستورات برنامه تمایز دهیم و خوب این کار با استثنائاتی از قبیل کاراکتر رشته وسط یک رسته و ... همراه خواهد بود که برای حل مشکل آن از اسلش استفاده می کنیم یا برای نمایش دادن کاراکتر اسلش از خود اسلش برای اسکیپ کردن استفاده می کنیم.

رشته ها و کاراکتر ها با اعلامی شروع می شوند، که شامل موارد زیر می شود :

“.....” -> String , ‘.....’ -> Character

دیاگرام گذار



عبارت منظم

“ (String* (\”)* String*)* “

String -> All String - “

“ (\” | String) ‘

String -> All String - ‘

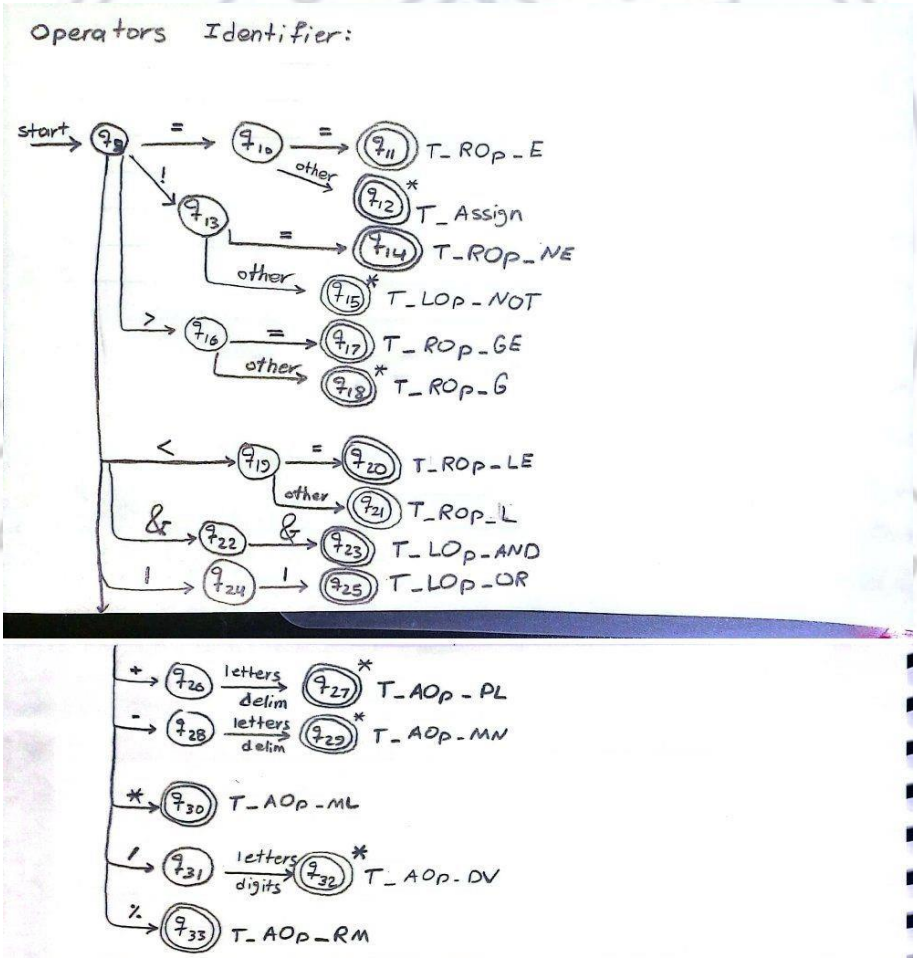
- operator identifier

در این قسمت قصد داریم توکن های مربوط به اپراتور ها و وضعیت هایی که قرار است در آن یک وضعیت را مانند مقایسه و ... پیاده سازی کنیم، که هر یکسری از آن ها در عمل بر یک دیگری تقدم دارد و این مورد را هنگام پیاده سازی باید بررسی و اعمال نمود.

اپراتور ها و عملگر ها شامل موارد زیر می شود :

= , == , ! , != , > , >= , < , <= , && , || , + , - , * , / , %

دیاگرام گذار



عبارت منظم

=

==

!= , !

>= , >

<= , <

&&

||

+

-

*

/

%

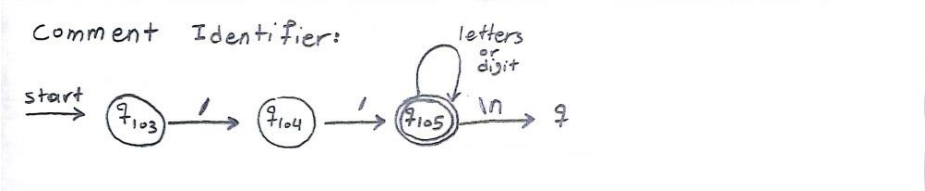
- comment identifier

در این قسمت قصد داریم توکن های مربوط کامنت را استخراج کنیم، که کامنت ها یکی از مهم ترین بخش های برنامه نوشته شده به حساب می آید، به علت اینکه راهنمایی محسوب می شود برای کسی که قرار است برنامه شده را بخواند.

کامنت شامل مورد زیر می شود :

//

دیاگرام گذار



عبارت منظم

به دلیل اینکه فقط از دو کاراکتر اسلش تشکیل شده، عبارت منظمش مثل همان توکن می باشد.

//

پایان

تحلیل گر لغوی

موفق باشید